Ch4.2: Dự báo tuổi thọ còn lại của động cơ bằng mạng MLP

Bài toán **hồi quy**: dự báo tuổi thọ còn lại (RUL, số chu trình còn lại đến lúc hỏng) của động cơ tuabin phản lực từ số đọc cảm biến, trên tập con FD001 của bộ dữ liệu C-MAPSS (NASA).

Tập huấn luyện gồm 100 động cơ chạy đến khi hỏng, được chia theo **động cơ** (không chia theo dòng) thành 80 động cơ huấn luyện và 20 động cơ kiểm định, để các chu trình của cùng một động cơ không rơi vào hai tập khác nhau. Tập kiểm tra gồm 100 động cơ của tệp `test_FD001.txt`, mỗi động cơ lấy chu trình cuối. Sai số đo bằng RMSE và MAE, đơn vị là chu trình.

Mỗi mạng được huấn luyện lại với ba hạt giống (2, 8, 24) và báo cáo bằng trung bình ± độ lệch chuẩn. Trong thực tế nên chạy nhiều hạt giống hơn (thường từ 5 đến 10) để đánh giá đúng độ ổn định của kết quả; ở đây là bài tập nên chỉ dùng 3 hạt giống để tiết kiệm thời gian chạy.

Chuẩn bị dữ liệu. Tải bộ Turbofan Engine Degradation Simulation Data Set từ Prognostics Center of Excellence của NASA (https://data.phmsociety.org/nasa/, mục 6), giải nén hai lượt rồi đặt các tệp .txt vào thư mục `CMAPSSData` trên Google Drive.

Mục	Nội dung
1	Chuẩn bị
2	Xác định bài toán, dữ liệu
3	Chia tập và tiền xử lý
4	Khai báo và huấn luyện mạng cơ bản
5	Cải tiến theo phương pháp thử và sai (Trial and Error)
6	Tinh chỉnh siêu tham số
7	Lưu và dùng lại mô hình
8	Tổng kết quy trình

1. Chuẩn bị

Nạp thư viện và thiết lập hình vẽ

Ô này nạp các thư viện dùng chung: `numpy` (tính toán mảng), `matplotlib` (vẽ hình) và `tensorflow` (xây dựng mạng nơ-ron), rồi in phiên bản TensorFlow đang chạy. `plt.rcParams` đặt font, độ phân giải và bỏ khung trên, khung phải cho mọi hình; từ điển `COLORS` giữ bảng màu thống nhất cho các hình ở các mục sau.

In [ ]:

```
import os, time
import numpy as np
```

```
import matplotlib.pyplot as plt
import tensorflow as tf

print("TensorFlow", tf.__version__)
plt.rcParams.update({"font.family": "DejaVu Sans", "font.size": 11,
                     "figure.dpi": 150, "axes.spines.top": False,
                     "axes.spines.right": False})
COLORS = {"grey": "#6E6D69", "purple": "#4A3AA7", "yellow": "#EDA100",
          "blue": "#2A78D6", "orange": "#EB6834", "red": "#E34948",
          "green": "#1BAF7A", "ink": "#0B0B0B"}
```

```
TensorFlow 2.21.0
```

2. Xác định bài toán, dữ liệu

2.1. Đọc dữ liệu C-MAPSS

Ô này kết nối Google Drive (`drive.mount`) và đọc ba tệp của tập con FD001: `train_FD001.txt` (động cơ chạy đến khi hỏng), `test_FD001.txt` (động cơ bị cắt giữa chừng) và `RUL_FD001.txt` (RUL thực tế tại chu trình cuối của từng động cơ trong tập kiểm tra). Mỗi dòng của hai tệp đầu là một chu trình bay với 26 cột: số hiệu động cơ `unit` , số chu trình `cycle` , ba thiết lập vận hành `setting1` – `setting3` và 21 cảm biến `s1` – `s21` ; hàm `read_data` đặt tên cột bằng `pandas.read_csv` . Kết quả: tập huấn luyện có 20.631 dòng, tập kiểm tra có 13.096 dòng (mỗi dòng 26 cột) và có 100 giá trị RUL thực tế, mỗi động cơ kiểm tra một giá trị.

In [ ]:

```
import numpy as np
import pandas as pd
from google.colab import drive

drive.mount("/content/gdrive")      # kết nối Google Drive
DATA_DIR = "/content/gdrive/MyDrive/Colab Notebooks/Data/CMAPSSData"  # Khai báo đúng đường dẫn thư mục

COLUMNS = ["unit", "cycle", "setting1", "setting2", "setting3"]
COLUMNS += [f"s{i}" for i in range(1, 22)]

def read_data(filename):
    return pd.read_csv(f"{DATA_DIR}/{filename}", sep=r"\s+",
                       header=None, names=COLUMNS)

# tập huấn luyện: 100 động cơ chạy đến khi hỏng
train_df = read_data("train_FD001.txt")
# tập kiểm tra: 100 động cơ bị cắt giữa chừng
test_df = read_data("test_FD001.txt")
rul_test = pd.read_csv(f"{DATA_DIR}/RUL_FD001.txt",
                       header=None)[0].values
print("Tập huấn luyện:", train_df.shape)
print("Tập kiểm tra  :", test_df.shape)
print("RUL kiểm tra  :", rul_test.shape)
```

```
Tập huấn luyện: (20631, 26)
Tập kiểm tra  : (13096, 26)
RUL kiểm tra  : (100,)
```

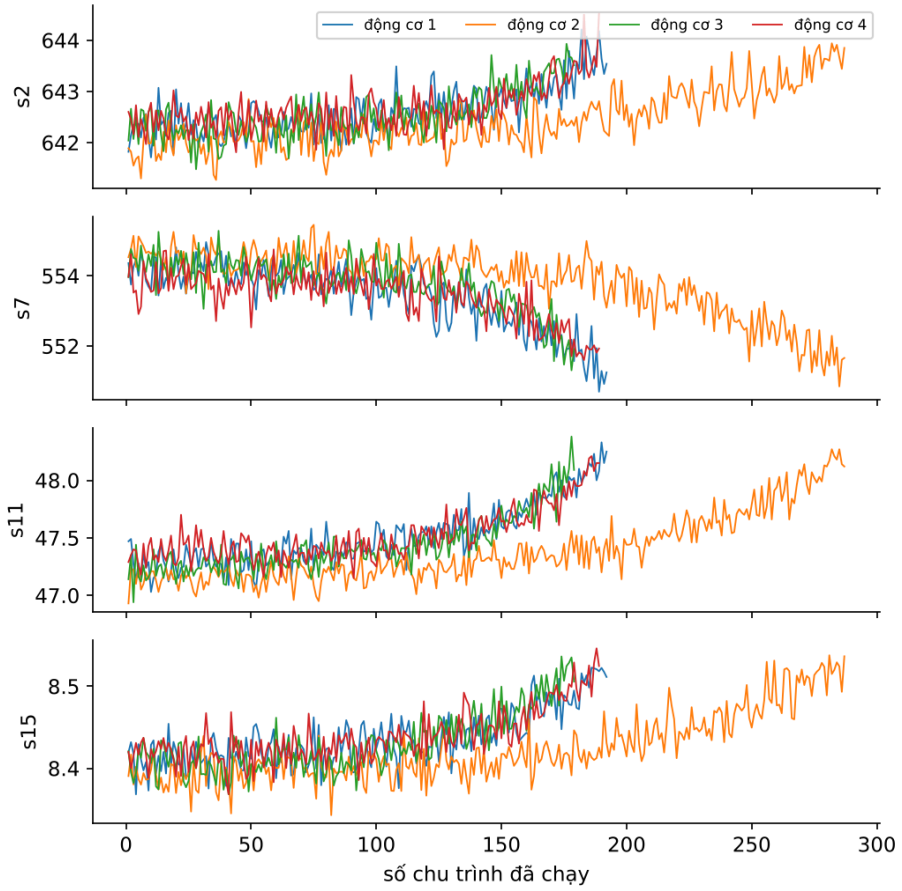
2.2. Quan sát diễn biến cảm biến

Ô này vẽ diễn biến bốn cảm biến (`s2` , `s7` , `s11` , `s15`) theo số chu trình trên bốn động cơ đầu tiên của tập huấn luyện, bằng `plt.subplots` với trục số chu trình chung. Kết quả: ở cả bốn động cơ, `s2` , `s11` và `s15` tăng dần còn `s7` giảm dần về cuối vòng đời, nhưng xu hướng chỉ rõ trong khoảng 60–80 chu trình cuối và bị nhiễu che khuất ở phần đầu. Động cơ 2 chạy gần 290 chu trình, lâu hơn hẳn ba động cơ còn lại (khoảng 190 chu trình), nên cùng một số chu trình đã chạy ứng với tuổi thọ còn lại rất khác nhau giữa các động cơ.

In [ ]:

```
import matplotlib.pyplot as plt

# Bốn cảm biến có xu hướng
PLOT_SENSORS = ["s2", "s7", "s11", "s15"]
fig, axes = plt.subplots(len(PLOT_SENSORS), 1, figsize=(7, 7),
                         sharex=True)
for ax, sensor in zip(axes, PLOT_SENSORS):
    for unit in [1, 2, 3, 4]:
        engine = train_df[train_df["unit"] == unit]
        ax.plot(engine["cycle"], engine[sensor], lw=0.9,
                label="động cơ %d" % unit)
    ax.set_ylabel(sensor)
axes[0].legend(fontsize=8, ncol=4)
axes[-1].set_xlabel("số chu trình đã chạy")
plt.tight_layout()
plt.show()
```



3. Chia tập và tiền xử lý

3.1. Tạo nhãn RUL và chọn đặc trưng

Ô này tạo nhãn cho tập huấn luyện. Với mỗi động cơ, RUL của một chu trình bằng số chu trình còn lại đến chu trình cuối của động cơ đó (`groupby("unit")` và `transform("max")`), rồi chặn trên ở `RUL_CAP` = 125 chu trình: khi động cơ còn rất xa lúc hỏng, cảm biến chưa cho dấu hiệu nào nên không có căn cứ phân biệt 200 hay 300 chu trình còn lại. Các cột có độ lệch chuẩn nhỏ hơn 10^{-4} (gần như không đổi trong suốt vòng đời) bị loại; 24 cột gồm thiết lập vận hành và cảm biến được xét, phần còn lại làm đặc trưng, lưu trong `FEATURES` . Kết quả: giữ lại 17 trong 24 cột làm đặc trưng.

In [ ]:

```
RUL_CAP = 125       # RUL sớm hơn mốc này coi như còn tốt

# Nhãn RUL: số chu trình còn lại đến lúc hỏng, chặn trên ở RUL_CAP
last_cycle = train_df.groupby("unit")["cycle"].transform("max")
cycles_left = last_cycle - train_df["cycle"]
train_df["RUL"] = cycles_left.clip(upper=RUL_CAP)

# Bỏ các cột gần như không đổi trong suốt vòng đời
FEATURES = [col for col in COLUMNS[2:]
            if train_df[col].std() > 1e-4]
print("Giữ lại", len(FEATURES), "trong", len(COLUMNS) - 2, "cột")
```

```
Giữ lại 17 trong 24 cột
```

3.2. Chia tập huấn luyện, kiểm định và kiểm tra theo động cơ

Ô này chia 100 động cơ huấn luyện thành 80 động cơ huấn luyện và 20 động cơ kiểm định bằng `train_test_split` trên danh sách số hiệu động cơ (`test_size=0.2` , `random_state=42`), rồi lấy toàn bộ chu trình của từng nhóm. Chia theo động cơ chứ không chia theo dòng, vì các chu trình liên tiếp của cùng một động cơ gần như giống nhau; chia theo dòng làm hai tập chứa chu trình lân cận nhau và đánh giá lạc quan quá mức. Tập kiểm tra là chu trình cuối của mỗi động cơ trong `test_df` (`groupby("unit").tail(1)`), nhãn lấy từ `RUL_FD001.txt` và cũng chặn ở `RUL_CAP` . Kết quả: 80 động cơ huấn luyện (16.561 dòng), 20 động cơ kiểm định (4.070 dòng) và 100 mẫu kiểm tra.

In [ ]:

```
from sklearn.model_selection import train_test_split

# Chia tập theo ĐỘNG CƠ, không chia theo dòng, để không rò rỉ
train_units, val_units = train_test_split(
    train_df["unit"].unique(), test_size=0.2, random_state=42)
train_set = train_df[train_df["unit"].isin(train_units)]
val_set = train_df[train_df["unit"].isin(val_units)]
# Chu trình cuối của mỗi động cơ trong tập kiểm tra
test_last = test_df.groupby("unit").tail(1)

y_train = train_set["RUL"].values
y_val = val_set["RUL"].values
y_test = np.minimum(rul_test, RUL_CAP)
```

```
print("Số động cơ  : huấn luyện", len(train_units),
      "| kiểm định", len(val_units), "| kiểm tra", len(test_last))
print("Số mẫu (dòng): huấn luyện", len(train_set),
      "| kiểm định", len(val_set), "| kiểm tra", len(test_last))
```

```
Số động cơ  : huấn luyện 80 | kiểm định 20 | kiểm tra 100
Số mẫu (dòng): huấn luyện 16561 | kiểm định 4070 | kiểm tra 100
```

3.3. Chuẩn hoá đặc trưng

Ô này dùng `StandardScaler` đưa mỗi đặc trưng về trung bình 0 và độ lệch chuẩn 1. Tham số chuẩn hoá chỉ ước lượng trên tập huấn luyện rồi áp cho cả ba tập, để thông tin của tập kiểm định và kiểm tra không lọt vào quá trình huấn luyện. Kết quả: ba mảng đặc trưng đều có 17 cột, với 16.561 dòng huấn luyện, 4.070 dòng kiểm định và 100 dòng kiểm tra.

In [ ]:

```
from sklearn.preprocessing import StandardScaler
from tensorflow import keras

# Tham số chuẩn hoá chỉ ước lượng trên tập huấn luyện
scaler = StandardScaler().fit(train_set[FEATURES])

X_train = scaler.transform(train_set[FEATURES])
X_val   = scaler.transform(val_set[FEATURES])
X_test  = scaler.transform(test_last[FEATURES])
print("Train:      ", X_train.shape)
print("Validation: ", X_val.shape)
print("Test:       ", X_test.shape)
```

```
Train:       (16561, 17)
Validation:  (4070, 17)
Test:        (100, 17)
```

4. Khai báo và huấn luyện mạng cơ bản

Mục này gộp ba việc đi liền nhau của quy trình: khai báo mạng, khai báo cấu hình huấn luyện và huấn luyện mạng cơ bản để chẩn đoán. Gồm năm mục: 4.1 định nghĩa mạng cơ sở 17–8–1; 4.2 khai báo cấu hình huấn luyện; 4.3 huấn luyện mạng với một hạt giống; 4.4 chẩn đoán bằng đường học và đồ thị RUL dự báo – thực tế; 4.5 huấn luyện với ba hạt giống để có mốc so sánh cho các mục sau.

4.1. Định nghĩa mạng cơ sở 17–8–1

Ô này dựng mạng cơ sở bằng `keras.Sequential`: lớp vào 17 đặc trưng, một lớp ẩn `Dense` 8 nơ-ron ReLU và lớp ra `Dense` 1 nơ-ron không có hàm kích hoạt, vì bài toán hồi quy cần đầu ra là một số thực không bị chặn. `set_random_seed(2)` cố định khởi tạo. Mạng `model_bs` chưa huấn luyện; mục 4.3 huấn luyện trực tiếp mạng này, còn mục 4.5 dùng nó làm khuôn, mỗi lần huấn luyện tạo bản sao mới bằng `clone_model`. Kết quả từ `model_bs.summary()`: 153 tham số, gồm 144 ở lớp ẩn và 9 ở lớp ra.

In [ ]:

```
# Định nghĩa mạng cơ sở 17-8-1 (model_bs): mạng chưa huấn luyện, dùng làm khuôn.
keras.utils.set_random_seed(2)
model_bs = keras.Sequential([
    keras.layers.Input(shape=(len(FEATURES),)),
    keras.layers.Dense(8, activation="relu"),    # lớp ẩn
    keras.layers.Dense(1),      # 1 nơ-ron, không hàm kích hoạt
])
model_bs.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8)	144
dense_1 (Dense)	(None, 1)	9

Total params: 153 (612.00 B)
Trainable params: 153 (612.00 B)
Non-trainable params: 0 (0.00 B)

4.2. Cấu hình huấn luyện (khai báo theo trình tự chung)

Ô này khai báo cách huấn luyện bằng `model_bs.compile`: bộ tối ưu Adam với tốc độ học 10^{-3}, hàm mất mát `mse` (sai số bình phương trung bình, phù hợp với bài toán hồi quy) và hai phép đo chỉ để theo dõi: RMSE (`RootMeanSquaredError`) và MAE. Callback `EarlyStopping` theo dõi mất mát kiểm định với patience 10 và khôi phục trọng số tốt nhất. Ô chỉ khai báo, chưa in kết quả.

Đây là khai báo theo trình tự chung của quy trình: cấu hình biên dịch gắn với chính mô hình `model_bs` và `early_stopping` chỉ được dùng khi gọi `fit()`. Trong bài này cấu hình được dùng ngay ở mục 4.3 qua `model_bs.fit(...)`. Từ mục 4.5 trở đi, mạng được tạo mới bằng `clone_model` (bản sao không mang theo thiết lập biên dịch) hoặc `build_network`, nên được biên dịch lại trong `train_once`, `build_network` và `run_variant`; muốn đổi tốc độ học hay patience ở những mục đó phải sửa tại những hàm này, không sửa ở mục 4.2.

In [ ]:

```
# Bộ tối ưu Adam, hàm mất mát MSE, phép đo RMSE và MAE, và dừng sớm.

model_bs.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="mse",      # RMSE và MAE chỉ để theo dõi
    metrics=[keras.metrics.RootMeanSquaredError(name="rmse"),
             "mae"],
)

early_stopping = keras.callbacks.EarlyStopping(
    monitor="val_loss", patience=10, restore_best_weights=True)
```

4.3. Huấn luyện mạng cơ sở với một hạt giống

Ô này huấn luyện trực tiếp mạng cơ sở `model_bs` bằng cấu hình đã khai báo ở mục 4.2: `model_bs.fit` với dữ liệu huấn luyện, dữ liệu kiểm định `(X_val, y_val)`, tối đa 200 epoch, lô 64 mẫu và callback `early_stopping`. Hạt giống 2 đã đặt khi định nghĩa mạng ở mục 4.1. Giá trị trả về của `fit` được lấy bằng `history.history` để vẽ đường học ở ô sau, rồi `model_bs.evaluate` tính

RMSE trên hai tập. Kết quả chính: RMSE huấn luyện 18,15 chu trình, RMSE kiểm định 16,70 chu trình và mạng chạy đủ 200 epoch mà dừng sớm không kích hoạt. Kết quả này chỉ từ một hạt giống nên chưa đủ để kết luận; mục 4.5 lặp lại việc huấn luyện với nhiều hạt giống.

In [ ]:

```
# Huấn luyện trực tiếp model_bs bằng cấu hình đã khai báo ở mục 4.2
# (hạt giống 2, đã đặt khi định nghĩa mạng ở mục 4.1)
history = model_bs.fit(
    X_train, y_train, validation_data=(X_val, y_val),
    epochs=200, batch_size=64, verbose=0,
    callbacks=[early_stopping])
history = history.history

print("RMSE huấn luyện:",
      round(model_bs.evaluate(X_train, y_train, verbose=0)[1], 2),
      "chu trình")
print("RMSE kiểm định :",
      round(model_bs.evaluate(X_val, y_val, verbose=0)[1], 2),
      "chu trình")
print("Số epoch đã chạy:", len(history["loss"]))
```

```
RMSE huấn luyện: 18.15 chu trình
RMSE kiểm định : 16.7 chu trình
Số epoch đã chạy: 200
```

4.4. Chẩn đoán bằng đường học và đồ thị RUL dự báo – thực tế

Ô này lấy mạng `model_bs` vừa huấn luyện ở mục 4.3 và vẽ hai hình: đường mất mát huấn luyện, kiểm định theo epoch, và đồ thị tán xạ RUL dự báo theo RUL thực tế trên tập kiểm định (đường nét đứt là dự báo lý tưởng). Hàm `rmse` tính căn bậc hai của sai số bình phương trung bình. Vòng lặp phía dưới tách tập kiểm định thành hai nhóm: động cơ gần hỏng (RUL < 50) và còn xa lúc hỏng (RUL ≥ 50), rồi in RMSE và sai số trung bình (dự báo trừ thực tế) của từng nhóm; sai số trung bình dương nghĩa là mạng dự báo lạc quan, tức cho rằng động cơ còn sống lâu hơn thực tế. Kết quả chính: mất mát giảm nhanh trong khoảng 25 epoch đầu rồi gần như phẳng, và hai đường sát nhau nên mạng chưa quá khớp. RMSE kiểm định (16,70) nhỏ hơn RMSE huấn luyện (18,15) vì nhóm 20 động cơ kiểm định cho sai số thấp hơn nhóm 80 động cơ huấn luyện, không phải dấu hiệu mạng dự báo tốt hơn trên dữ liệu mới. Đồ thị tán xạ cho thấy mạng nén dự báo về khoảng 100–120 chu trình: các điểm nằm phía trên đường lý tưởng khi RUL thực tế từ khoảng 20 đến 100 chu trình, tức mạng dự báo lạc quan. Số liệu theo khoảng RUL khớp với nhận xét này: nhóm gần hỏng (RUL < 50) có RMSE 14,28 chu trình và sai số trung bình +5,6 chu trình, nhóm còn xa (RUL ≥ 50) có RMSE 17,41 chu trình và sai số trung bình −2,7 chu trình. Dự báo lạc quan ở động cơ gần hỏng là loại sai số nguy hiểm hơn cả, vì nó trì hoãn việc bảo trì (số liệu từ một hạt giống nên thay đổi theo từng lần chạy).

In [ ]:

```
import matplotlib.pyplot as plt

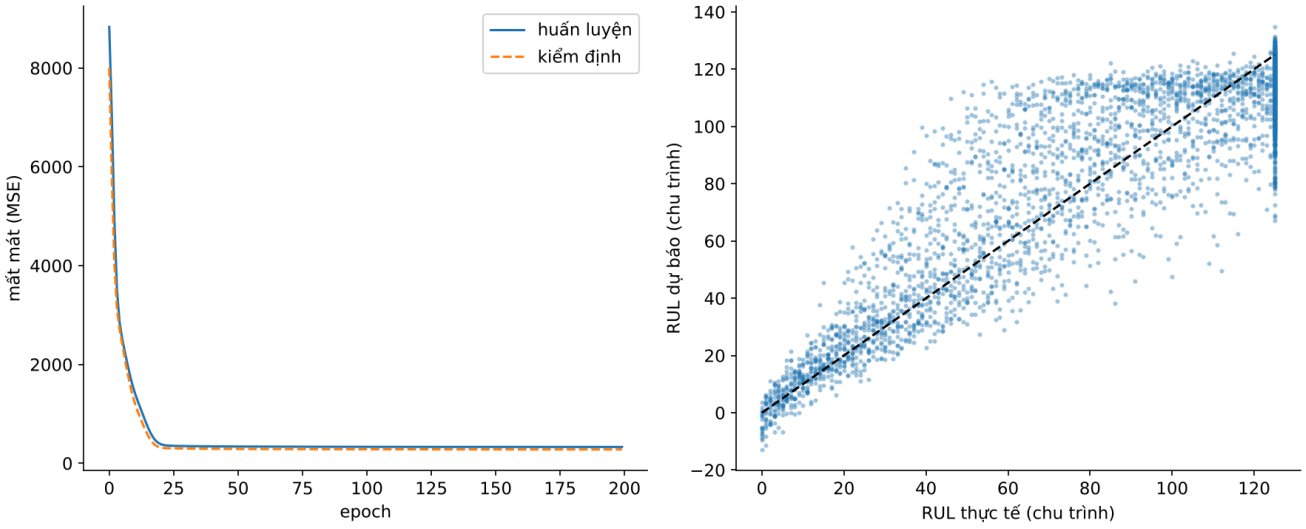
def rmse(y_true, y_pred):
    return float(np.sqrt(np.mean((y_pred - y_true) ** 2)))

y_pred = model_bs.predict(X_val, verbose=0).ravel()

fig, (ax_loss, ax_scatter) = plt.subplots(1, 2, figsize=(12, 5))
```

```
ax_loss.plot(history["loss"], label="huấn luyện")
ax_loss.plot(history["val_loss"], "--", label="kiểm định")
ax_loss.set_xlabel("epoch"); ax_loss.set_ylabel("mất mát (MSE)")
ax_loss.legend()
ax_scatter.scatter(y_val, y_pred, s=4, alpha=0.3)
ax_scatter.plot([0, RUL_CAP], [0, RUL_CAP], "k--")    # lý tưởng
ax_scatter.set_xlabel("RUL thực tế (chu trình)")
ax_scatter.set_ylabel("RUL dự báo (chu trình)")
plt.tight_layout()
plt.show()

# Sai số theo khoảng RUL: gần hỏng và còn xa
RANGES = (("RUL < 50 ", y_val < 50), ("RUL >= 50", y_val >= 50))
for name, mask in RANGES:
    error = y_pred[mask] - y_val[mask]
    print(f"{name}: RMSE {rmse(y_val[mask], y_pred[mask]):.2f} | "
          f"sai số trung bình {error.mean():+.1f} chu trình")
```




```
RUL < 50 : RMSE 14.28 | sai số trung bình +5.6 chu trình
RUL >= 50: RMSE 17.41 | sai số trung bình -2.7 chu trình
```

4.5. Huấn luyện mạng cơ sở với ba hạt giống

Ô này huấn luyện lại mạng cơ sở ba lần, mỗi lần một hạt giống (2, 8, 24). Trong thực tế nên chạy nhiều hạt giống hơn (thường từ 5 đến 10) để đánh giá đúng độ ổn định của kết quả; ở đây là bài tập nên giới hạn 3 hạt giống để tiết kiệm thời gian. Hàm `train_once` tạo mạng mới bằng `clone_model(model_bs)`, biên dịch lại với cùng cấu hình ở mục 4.2, huấn luyện tối đa 200 epoch (lô 64 mẫu, dừng sớm), rồi ô tính RMSE huấn luyện, RMSE kiểm định và MAE kiểm định cho từng hạt giống và gom vào bảng `pandas` kèm trung bình và độ lệch chuẩn. Hạt giống 2 cho lại đúng kết quả của mục 4.3. Kết quả chính: RMSE huấn luyện 18,35 ± 0,26 chu trình, RMSE kiểm định 16,96 ± 0,26 chu trình, MAE kiểm định 12,65 ± 0,36 chu trình; hai lần chạy đủ 200 epoch mà dừng sớm không kích hoạt, riêng hạt giống 8 dừng ở epoch 187. Hạt giống 2 tốt nhất (RMSE kiểm định 16,70) và hạt giống 24 kém nhất (17,31): chênh lệch giữa các hạt giống đã hơn nửa chu trình (0,61).

In [ ]:

```
import statistics
import pandas as pd

SEEDS = (2, 8, 24)    # ba hạt giống; trong thực tế nên dùng nhiều hơn
```

```
def train_once(seed):
    keras.utils.set_random_seed(seed)
    # mạng mới có cùng kiến trúc với model_bs, trọng số khởi tạo lại
    model = keras.models.clone_model(model_bs)
    model.compile(optimizer=keras.optimizers.Adam(1e-3),
                  loss="mse",
                  metrics=[keras.metrics.RootMeanSquaredError(
                      name="rmse"), "mae"])
    early_stopping = keras.callbacks.EarlyStopping(
        monitor="val_loss", patience=10,
        restore_best_weights=True)
    history = model.fit(
        X_train, y_train, validation_data=(X_val, y_val),
        epochs=200, batch_size=64, verbose=0,
        callbacks=[early_stopping])
    return model, history.history

runs = [train_once(seed) for seed in SEEDS]
train_rmse, val_rmse, val_mae, epochs = [], [], [], []
for model, history in runs:
    train_rmse.append(model.evaluate(X_train, y_train,
                                     verbose=0)[1])
    val_scores = model.evaluate(X_val, y_val, verbose=0)
    val_rmse.append(val_scores[1])
    val_mae.append(val_scores[2])
    epochs.append(len(history["loss"]))

# Bảng kết quả từng hạt giống, kèm trung bình và độ lệch chuẩn
per_seed = pd.DataFrame({
    "RMSE huấn luyện (chu trình)": train_rmse,
    "RMSE kiểm định (chu trình)": val_rmse,
    "MAE kiểm định (chu trình)": val_mae,
    "Số epoch": epochs}, index=pd.Index(SEEDS, name="Hạt giống"))
summary = pd.DataFrame({
    "Trung bình": per_seed.mean(),
    "Độ lệch chuẩn (±)": per_seed.apply(statistics.pstdev)}).T
display(pd.concat([per_seed, summary]).round(2))
```

	RMSE huấn luyện (chu trình)	RMSE kiểm định (chu trình)	MAE kiểm định (chu trình)
2	18.15	16.70	12.30
8	18.18	16.86	12.52
24	18.72	17.31	13.14
Trung bình	18.35	16.96	12.65
Độ lệch chuẩn (±)	0.26	0.26	0.36

5. Cải tiến theo phương pháp thử và sai (Trial and Error)

5.1. Phương pháp thử và sai (Trial and Error) và cách làm ở mục 5

Thử và sai là phương pháp cải tiến mô hình bằng thí nghiệm: mỗi lần chỉ thay đổi một yếu tố, huấn luyện lại, đo kết quả trên tập kiểm định, giữ lại thay đổi có ích và bỏ thay đổi không hiệu quả. Khác với dò tìm ngẫu nhiên ở mục 6, các thử nghiệm ở đây không chọn tuỳ ý mà đi theo chẩn đoán ở mục 4: đường mất mát phẳng sau khoảng 25 epoch và hai đường sát nhau cho thấy mạng chưa quá khớp, còn đồ thị RUL dự báo – thực tế cho thấy mạng dự báo lạc quan ở khoảng giữa vòng đời.

Nguyên nhân có thể là mạng chưa được huấn luyện đủ lâu hoặc mạng quá nhỏ, nên hai phương án đầu tiên lần lượt kiểm tra hai khả năng này; quá khớp, nếu phát sinh sau khi mở rộng mạng, được xử lý sau.

Mục 5 gồm ba nhóm phương án nối tiếp nhau. Mục 5.4 chỉ kéo dài thời gian huấn luyện (tối đa 2000 epoch, để dừng sớm quyết định điểm dừng) và giữ nguyên mạng. Mục 5.5 tăng kích thước mạng lên 17–256–256–1. Mục 5.6 thêm từng kỹ thuật chính quy hoá vào mạng mở rộng (L2, Dropout, dừng sớm, và BatchNormalization để đối chiếu), mỗi lần chỉ thêm một kỹ thuật để thấy tác dụng riêng, rồi gộp cả bốn. Mỗi phương án được huấn luyện với cả ba hạt giống và báo cáo bằng trung bình ± độ lệch chuẩn, vì chênh lệch nhỏ giữa hai phương án có thể chỉ do khởi tạo ngẫu nhiên. Mọi so sánh đều dùng tập kiểm định; tập kiểm tra được để dành cho đánh giá cuối ở mục 6. Cuối mục có bảng tổng hợp tám phương án để chọn hướng cải tiến.

5.2. Hàm `build_network`: dựng mạng có công tắc cho từng kỹ thuật

Hàm `build_network` dựng mạng MLP hồi quy 17–hidden–1 dùng chung cho mọi phương án của mục 5. Mỗi kỹ thuật được bật hoặc tắt bằng một đối số: `hidden` (số nơ-ron của từng lớp ẩn), `batch_norm` (lớp `BatchNormalization` đặt giữa lớp tuyến tính và hàm kích hoạt), `dropout` (lớp `Dropout`), `l2` (phạt L2 qua `kernel_regularizer`) và `learning_rate`. Nhờ vậy hai phương án bất kỳ chỉ khác nhau đúng một thành phần. Ô này chỉ định nghĩa hàm, chưa in kết quả.

In [ ]:

```
def build_network(hidden=(8,), batch_norm=False, dropout=0.0,
                  l2=0.0, learning_rate=1e-3):
    """Mạng MLP hồi quy 17-hidden-1; mỗi đối số bật một kỹ thuật."""
    regularizer = keras.regularizers.l2(l2) if l2 else None
    layers = [keras.layers.Input(shape=(len(FEATURES),))]
    for units in hidden:
        layers.append(keras.layers.Dense(
            units, activation=None if batch_norm else "relu",
            kernel_regularizer=regularizer))
        if batch_norm:      # đặt giữa lớp tuyến tính và kích hoạt
            layers += [keras.layers.BatchNormalization(),
                       keras.layers.Activation("relu")]
        if dropout:
            layers.append(keras.layers.Dropout(dropout))
    layers.append(keras.layers.Dense(1))
    model = keras.Sequential(layers)
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate),
        loss="mse",
        metrics=[keras.metrics.RootMeanSquaredError(name="rmse")])
    return model
```

5.3. Hàm `run_variant` và `show_results`: chạy ba hạt giống và tổng hợp kết quả

Hàm `run_variant` huấn luyện một phương án với ba hạt giống, mỗi hạt giống một mạng mới dựng từ `build_network`, rồi lưu RMSE huấn luyện, RMSE kiểm định, số epoch và đường mất mát vào từ điển `RESULTS`. Đối số `early_stop` bật `EarlyStopping` (patience 10, khôi phục trọng số tốt nhất), `epochs` đặt số epoch tối đa (mặc định 200). Hàm này không in gì. Hàm `summarize` gom kết quả của một phương án thành một dòng gồm RMSE huấn luyện, RMSE kiểm định (trung bình ± độ lệch

chuẩn qua ba hạt giống), độ chênh và số epoch trung bình; hàm `show_results` hiển thị các dòng đó thành bảng `pandas.DataFrame`, và mỗi phương án của mục 5 kết thúc bằng một lần gọi hàm này. Độ chênh bằng RMSE kiểm định trừ RMSE huấn luyện (chu trình): chênh dương lớn là dấu hiệu quá khớp; chênh âm hoặc gần 0 nghĩa là chưa quá khớp (ở bài này độ chênh âm vì nhóm động cơ kiểm định có sai số thấp hơn nhóm huấn luyện).

In [ ]:

```
RESULTS = {}

def run_variant(label, early_stop=False, epochs=200, **options):
    """Huấn luyện ba hạt giống; đo trên tập kiểm định."""
    rows = []
    for seed in SEEDS:
        keras.utils.set_random_seed(seed)
        model = build_network(**options)
        callbacks = [keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=10,
            restore_best_weights=True)] if early_stop else []
        history = model.fit(
            X_train, y_train, validation_data=(X_val, y_val),
            epochs=epochs, batch_size=64, verbose=0,
            callbacks=callbacks)
        rows.append(dict(
            train=model.evaluate(X_train, y_train, verbose=0)[1],
            val=model.evaluate(X_val, y_val, verbose=0)[1],
            epochs=len(history.history["loss"]),
            val_loss=history.history["val_loss"],
            loss=history.history["loss"]))
    RESULTS[label] = rows


def summarize(label):
    """Một dòng kết quả: trung bình qua ba hạt giống."""
    rows = RESULTS[label]
    train = [r["train"] for r in rows]
    val = [r["val"] for r in rows]
    return {"Phương án": label,
            "RMSE huấn luyện": round(np.mean(train), 2),
            "RMSE kiểm định":
                f"{np.mean(val):.2f} ± {np.std(val):.2f}",
            "Chênh (chu trình)": round(np.mean(val) - np.mean(train), 2),
            "Số epoch": round(np.mean([r["epochs"] for r in rows]))}

def show_results(labels):
    """Hiển thị bảng kết quả của các phương án đã chạy."""
    display(pd.DataFrame([summarize(label) for label in labels]))
```

5.4. Huấn luyện lâu hơn

Chẩn đoán ở mục 4 cho thấy mạng 17–8–1 chạy đủ 200 epoch mà dừng sớm không kích hoạt, nên thí nghiệm đầu tiên chỉ đổi thời gian huấn luyện: giữ nguyên mạng, cho tối đa 2000 epoch và để dừng sớm quyết định điểm dừng. Mốc so sánh là cùng mạng huấn luyện 200 epoch. Kết quả chính: RMSE kiểm định gần như không đổi, từ 16,96 ± 0,26 xuống 16,94 ± 0,24 chu trình, dù mạng chạy lâu hơn (trung bình 335 epoch so với 200). Huấn luyện lâu hơn không giúp, nên giới hạn nằm ở

dung lượng của mạng chứ không ở thời gian huấn luyện. Kết quả được hiển thị thành bảng gồm RMSE huấn luyện, RMSE kiểm định (trung bình ± độ lệch chuẩn), độ chênh kiểm định – huấn luyện và số epoch trung bình.

In [ ]:

```
# Mạng cơ sở 17-8-1: 200 epoch (mốc so sánh) và huấn luyện lâu hơn
run_variant("17-8-1", hidden=(8,))
run_variant("17-8-1, huấn luyện lâu", early_stop=True, epochs=2000,
            hidden=(8,))
show_results(["17-8-1", "17-8-1, huấn luyện lâu"])
```

	Phương án	RMSE huấn luyện	RMSE kiểm định	Chênh (chu trình)
0	17-8-1	18.35	16.96 ± 0.26	-1.39
1	17-8-1, huấn luyện lâu	18.34	16.94 ± 0.24	-1.40

5.5. Tăng kích thước mạng

Mạng mở rộng 17–256–256–1 (70.657 tham số, so với 153 tham số của mạng cơ sở) được huấn luyện 200 epoch, cùng ngân sách với mốc ở mục 5.4. Kết quả chính: RMSE huấn luyện giảm từ 18,35 xuống 15,44 chu trình nhưng RMSE kiểm định tăng từ 16,96 lên 18,91 ± 0,26 chu trình; độ chênh đổi từ −1,39 sang +3,47 chu trình, tức mạng đã quá khớp. Tăng dung lượng một mình không đủ, cần chính quy hoá. Kết quả được hiển thị thành bảng gồm RMSE huấn luyện, RMSE kiểm định (trung bình ± độ lệch chuẩn), độ chênh kiểm định – huấn luyện và số epoch trung bình.

In [ ]:

```
# Mạng mở rộng 17-256-256-1 (200 epoch)
run_variant("17-256-256-1", hidden=(256, 256))
show_results(["17-256-256-1"])
```

	Phương án	RMSE huấn luyện	RMSE kiểm định	Chênh (chu trình)
0	17-256-256-1	15.44	18.91 ± 0.26	3.47

5.6. Chính quy hoá mạng mở rộng

Ba kỹ thuật chính quy hoá – L2 (hệ số 10^{-3}), Dropout (0,3) và dừng sớm – được thử riêng lẻ trên mạng 17–256–256–1, cùng với BatchNormalization (kỹ thuật ổn định huấn luyện, không thuộc nhóm chính quy hoá, đưa vào để đối chiếu), rồi gộp cả bốn với hệ số L2 giảm xuống 10^{-4}. Kết quả chính: dừng sớm đem lại hiệu quả lớn nhất, đưa RMSE kiểm định từ 18,91 xuống 16,99 ± 0,30 chu trình chỉ sau 32 epoch và xoá hẳn quá khớp (độ chênh −0,97). Dropout cũng có tác dụng rõ (17,32 ± 0,19; độ chênh −0,05), còn L2 với hệ số 10^{-3} chỉ cải thiện ít (18,75 ± 0,05; độ chênh vẫn +3,32). BatchNormalization không cải thiện mà còn kém hơn mạng chưa chính quy hoá (20,68 ± 0,48), phù hợp với vai trò của nó là ổn định huấn luyện chứ không giảm quá khớp. Phương án gộp cả bốn kỹ thuật đạt 16,80 ± 0,06 chu trình sau 40 epoch, thấp nhất trong các phương án, nhưng chỉ hơn dừng sớm đứng riêng 0,19 chu trình, nhỏ hơn độ lệch chuẩn của dừng sớm (0,30). Kết quả được hiển thị thành bảng gồm RMSE huấn luyện, RMSE kiểm định (trung bình ± độ lệch chuẩn), độ chênh kiểm định – huấn luyện và số epoch trung bình.

In [ ]:

```
# Mạng mở rộng 17-256-256-1, mỗi lần bật thêm MỘT kỹ thuật
WIDE = dict(hidden=(256, 256))
run_variant("+ L2 1e-3", l2=1e-3, **WIDE)
run_variant("+ Dropout 0,3", dropout=0.3, **WIDE)
run_variant("+ BatchNormalization", batch_norm=True, **WIDE)
run_variant("+ dừng sớm", early_stop=True, **WIDE)
run_variant("+ cả bốn kỹ thuật", early_stop=True, batch_norm=True,
            dropout=0.3, l2=1e-4, **WIDE)
show_results(["+ L2 1e-3", "+ Dropout 0,3", "+ BatchNormalization",
              "+ dừng sớm", "+ cả bốn kỹ thuật"])
```

	Phương án	RMSE huấn luyện	RMSE kiểm định	Chênh (chu trình)
0	+ L2 1e-3	15.43	18.75 ± 0.05	3.32
1	+ Dropout 0,3	17.36	17.32 ± 0.19	-0.05
2	+ BatchNormalization	16.07	20.68 ± 0.48	4.61
3	+ dừng sớm	17.96	16.99 ± 0.30	-0.97
4	+ cả bốn kỹ thuật	17.89	16.80 ± 0.06	-1.09

5.7. Tổng hợp: bảng tám phương án

Ô này tổng hợp tám phương án thành một bảng `pandas.DataFrame`. Ô này gọi lại `show_results` (dùng `summarize` ở ô định nghĩa `run_variant`) cho cả tám phương án theo thứ tự thử nghiệm, với các cột RMSE huấn luyện, RMSE kiểm định (trung bình ± độ lệch chuẩn qua ba hạt giống), độ chênh và số epoch trung bình. Kết quả chính: huấn luyện lâu hơn (mục 5.4) không cải thiện so với mạng cơ sở (16,94 so với 16,96 chu trình); mở rộng mạng (mục 5.5) làm quá khớp (18,91 chu trình, độ chênh +3,47); chính quy hoá (mục 5.6), nhất là dừng sớm, đưa RMSE kiểm định về 16,80 chu trình ở phương án gộp. So với mạng cơ sở 17–8–1, mức cải thiện tốt nhất chỉ là 0,16 chu trình, nhỏ hơn độ lệch chuẩn của mạng cơ sở (0,26), trong khi mạng lớn có số tham số gấp khoảng 460 lần (70.657 so với 153). Dung lượng và chính quy hoá chỉ cải thiện nhỏ, gợi ý rằng giới hạn nằm ở đầu vào chỉ gồm số đọc của một chu trình chứ không ở kiến trúc mạng.

In [ ]:

```
ORDER = ["17-8-1", "17-8-1, huấn luyện lâu", "17-256-256-1",
         "+ L2 1e-3", "+ Dropout 0,3", "+ BatchNormalization",
         "+ dừng sớm", "+ cả bốn kỹ thuật"]

show_results(ORDER)
```

	Phương án	RMSE huấn luyện	RMSE kiểm định	Chênh (chu trình)
0	17-8-1	18.35	16.96 ± 0.26	-1.39
1	17-8-1, huấn luyện lâu	18.34	16.94 ± 0.24	-1.40
2	17-256-256-1	15.44	18.91 ± 0.26	3.47
3	+ L2 1e-3	15.43	18.75 ± 0.05	3.32
4	+ Dropout 0,3	17.36	17.32 ± 0.19	-0.05
5	+ BatchNormalization	16.07	20.68 ± 0.48	4.61
6	+ dừng sớm	17.96	16.99 ± 0.30	-0.97
7	+ cả bốn kỹ thuật	17.89	16.80 ± 0.06	-1.09

6. Tinh chỉnh siêu tham số

6.1. Phương pháp làm ở mục 6: tìm kiếm siêu tham số và kiểm định chéo

Siêu tham số là những lựa chọn đặt trước khi huấn luyện và không học được từ dữ liệu, gồm số lớp ẩn, số nơ-ron mỗi lớp, tốc độ học, tỉ lệ Dropout, hệ số L2 và việc có dùng BatchNormalization hay không. Mục 5 chỉ thử một vài phương án theo chẩn đoán; mục 6 tìm có hệ thống trong cả không gian siêu tham số để chọn cấu hình tốt nhất.

Có hai cách tìm kiếm phổ biến. Tìm kiếm lưới (Grid Search) khai báo một danh sách giá trị cho từng siêu tham số rồi thử mọi tổ hợp. Cách này đầy đủ và dễ lặp lại, nhưng số tổ hợp tăng theo cấp số nhân với số siêu tham số (sáu siêu tham số, mỗi siêu tham số năm giá trị đã là hơn một vạn tổ hợp), và nó lặp đi lặp lại cùng vài giá trị của siêu tham số quan trọng trong khi vẫn tốn công cho siêu tham số ít ảnh hưởng. Tìm kiếm ngẫu nhiên (Random Search) bốc ngẫu nhiên giá trị của từng siêu tham số từ miền cho trước (dùng thang logarit cho tốc độ học và hệ số L2, vì các giá trị này trải qua nhiều bậc độ lớn) và chỉ thử một số cấu hình cố định theo ngân sách thời gian. Mỗi lần thử cho giá trị mới của mọi siêu tham số nên với cùng số lần huấn luyện, tìm kiếm ngẫu nhiên thường khám phá được nhiều hơn khi chỉ một vài siêu tham số thật sự quan trọng.

Mỗi cấu hình cần một điểm số đáng tin để so sánh. Nếu chỉ chia huấn luyện và kiểm định một lần, điểm số phụ thuộc vào nhóm động cơ kiểm định cụ thể và có thể may rủi. Kiểm định chéo (Cross-Validation) khắc phục điều này bằng cách huấn luyện và chấm điểm nhiều lần trên các cách chia khác nhau rồi lấy trung bình; chi tiết ở phần giải thích kiểm định chéo bên dưới.

Mục này gồm bốn việc và một bảng tổng hợp. Trước hết xác định các siêu tham số và khoảng giá trị tìm kiếm. Mục 6.6 minh hoạ tìm kiếm lưới với hai siêu tham số, mỗi siêu tham số ba giá trị, tức chín cấu hình. Mục 6.7 dùng tìm kiếm ngẫu nhiên trên tất cả các siêu tham số, thử mười cấu hình. Cuối cùng cấu hình tốt nhất của tìm kiếm ngẫu nhiên được huấn luyện lại và đánh giá trên tập kiểm tra. Mọi cấu hình đều được chấm bằng kiểm định chéo với ba hạt giống như ở mục 5, và việc tìm kiếm dùng thư viện Keras Tuner. Tập kiểm tra chỉ được dùng một lần, ở bước đánh giá cuối. Bảng tổng hợp ở cuối mục đặt năm phương án cạnh nhau, từ mạng nông đến mạng tinh chỉnh siêu tham số, để thấy mỗi bước cải tiến đóng góp bao nhiêu.

6.2. Keras Tuner: thư viện tìm kiếm siêu tham số của Keras

Keras Tuner (`keras_tuner`, thường nạp với tên `kt`) là thư viện do nhóm Keras phát triển để tìm siêu tham số cho mô hình Keras; trên Colab cần cài bằng `pip install keras-tuner`. Cách dùng gồm ba phần.

Phần thứ nhất là hàm dựng mạng nhận một đối số `hp` (đối tượng `HyperParameters`). Trong hàm, mỗi siêu tham số được khai báo bằng một lệnh: `hp.Int` (số nguyên trong một khoảng), `hp.Float` (số thực trong một khoảng, thêm `sampling="log"` để lấy theo thang logarit), `hp.Choice` (một giá trị trong danh sách) và `hp.Boolean` (bật hoặc tắt). Mỗi lần thử, tuner đưa vào `hp` một bộ giá trị cụ thể và hàm trả về mạng đã biên dịch. Đối số `default` là giá trị mà siêu tham số nhận khi nó không được đưa vào tìm kiếm.

Phần thứ hai là chọn tuner: `kt.GridSearch` thử mọi tổ hợp, `kt.RandomSearch` thử các tổ hợp ngẫu nhiên. Các đối số chính gồm `objective` (đại lượng cần tối ưu và hướng tối ưu, `"max"` hoặc `"min"`), `max_trials` (số cấu hình thử tối đa), `seed` (cố định lần bốc ngẫu nhiên), `directory` và

`project_name` (nơi lưu kết quả), `overwrite` (xoá kết quả cũ khi chạy lại). Phần thứ ba là gọi `tuner.search(...)` để chạy tìm kiếm rồi lấy cấu hình tốt nhất bằng `tuner.get_best_hyperparameters`.

Mặc định Keras Tuner chấm điểm mỗi cấu hình bằng một lần huấn luyện bằng `model.fit` và không có sẵn kiểm định chéo. Vì vậy bài này tạo lớp con của `kt.GridSearch` và `kt.RandomSearch`, ghi đè phương thức `run_trial` (phương thức mà tuner gọi cho mỗi cấu hình) để mỗi cấu hình được chấm bằng kiểm định chéo với ba hạt giống.

6.3. Xác định siêu tham số và khoảng giá trị tìm kiếm

Ô này nạp Keras Tuner và định nghĩa hai hàm. Hàm `layer_sizes` suy ra số nơ-ron từng lớp ẩn từ số lớp và số nơ-ron lớp đầu (mỗi lớp sau bằng một nửa lớp trước, tối thiểu 8). Hàm `build_model(hp)` khai báo sáu siêu tham số bằng `hp.Int`, `hp.Choice`, `hp.Boolean` và `hp.Float`, rồi gọi lại `build_network` của mục 5 để dựng mạng đã biên dịch. Khoảng tìm kiếm và giá trị mặc định như sau.

Siêu tham số	Khoảng tìm kiếm	Mặc định
`num_layers` (số lớp ẩn)	1, 2, 3	2
`units_1` (số nơ-ron lớp ẩn đầu)	16, 32, 64, 128, 256	64
`learning_rate` (tốc độ học)	từ 10^{-4} đến 10^{-2}, thang logarit	10^{-3}
`dropout`	0, 0,1, 0,2, 0,3	0,1
`l2` (hệ số L2)	từ 10^{-6} đến 10^{-3}, thang logarit	10^{-4}
`batch_norm`	tắt, bật	tắt

Ô chỉ định nghĩa hàm, chưa in kết quả.

In [ ]:

```
!pip install -q keras-tuner

import keras_tuner as kt

def layer_sizes(num_layers, units_1):
    """Số nơ-ron từng lớp ẩn: lớp sau bằng một nửa lớp trước."""
    return tuple(max(units_1 // 2 ** k, 8)
                 for k in range(num_layers))

def build_model(hp):
    """Dựng mạng từ một bộ siêu tham số hp do Keras Tuner đưa vào."""
    num_layers = hp.Int("num_layers", 1, 3, default=2)
    units_1 = hp.Choice("units_1", [16, 32, 64, 128, 256], default=64)
    return build_network(
        hidden=layer_sizes(num_layers, units_1),
        batch_norm=hp.Boolean("batch_norm", default=False),
        dropout=hp.Choice("dropout", [0.0, 0.1, 0.2, 0.3],
                          default=0.1),
        l2=hp.Float("l2", 1e-6, 1e-3, sampling="log", default=1e-4),
        learning_rate=hp.Float("learning_rate", 1e-4, 1e-2,
                               sampling="log", default=1e-3))
```

```
SPACE_NAMES = ["num_layers", "units_1", "learning_rate",
               "dropout", "l2", "batch_norm"]
```

6.4. Kiểm định chéo k-fold và GroupKFold

Kiểm định chéo k-fold chia dữ liệu thành k phần (fold). Lần lượt mỗi phần được giữ lại làm tập kiểm định, mạng được huấn luyện trên k − 1 phần còn lại, nên k lần cho k điểm số; điểm của cấu hình là trung bình của k điểm số kèm độ lệch chuẩn. Mọi mẫu được dùng để kiểm định đúng một lần, nên điểm số ít phụ thuộc vào cách chia; cái giá là mỗi cấu hình phải huấn luyện k lần.

Với dữ liệu vòng đời động cơ, các chu trình liên tiếp của cùng một động cơ gần như giống nhau. Nếu chia ngẫu nhiên từng dòng như `KFold` thông thường, các chu trình lân cận của một động cơ có thể rơi vào hai phía của phép chia và làm điểm số cao giả tạo. `GroupKFold` của scikit-learn giải quyết bằng cách chia theo nhóm: các mẫu cùng một nhóm luôn nằm cùng một phía. Ở đây nhóm là số hiệu động cơ (`unit`) và k = 4: 100 động cơ huấn luyện được chia thành bốn phần, mỗi phần 25 động cơ, và mỗi lần giữ lại trọn một phần. Lệnh `GroupKFold(n_splits=4).split(X, y, groups)` trả về bốn cặp chỉ số (huấn luyện, giữ lại). Tập kiểm tra gồm các động cơ của tệp `test_FD001.txt` , không tham gia kiểm định chéo.

Bộ chuẩn hoá `StandardScaler` được khớp lại trên phần huấn luyện của từng lần để dữ liệu giữ lại không ảnh hưởng đến mạng. Mỗi lần còn được lặp với ba hạt giống nên mỗi cấu hình tốn 4 × 3 = 12 lần huấn luyện.

6.5. Kiểm định chéo theo động cơ với ba hạt giống

Ô này chuẩn bị bốn phần cho kiểm định chéo (`GroupKFold` , nhóm là `unit` , dùng cả 100 động cơ huấn luyện) và định nghĩa cách chấm điểm. Lớp `CrossValidation` ghi đè `run_trial` : với mỗi cấu hình, vòng lặp chạy qua ba hạt giống và bốn phần; mỗi lần khớp lại `StandardScaler` , dựng mạng bằng `self.hypermodel.build(hp)` , huấn luyện tối đa 100 epoch với dừng sớm (patience 10) rồi đo RMSE trên phần giữ lại. `run_trial` trả về RMSE trung bình `cv_rmse` (đại lượng mà tuner tối thiểu hoá) và độ lệch chuẩn `cv_std` của 12 điểm số. Hai lớp `CVGridSearch` và `CVRandomSearch` kết hợp lớp này với hai tuner của Keras Tuner. Ô còn định nghĩa hai hàm: `trial_table` gom các lần thử thành bảng, và `describe_best` in cấu hình tốt nhất cùng điểm kiểm định chéo của nó. Ô chỉ định nghĩa, chưa in kết quả.

In [ ]:

```
from sklearn.model_selection import GroupKFold

# Kiểm định chéo theo ĐỘNG CƠ trên toàn bộ 100 động cơ huấn luyện
X_search = train_df[FEATURES].values
y_search = train_df["RUL"].values
folds = list(GroupKFold(n_splits=4).split(
    X_search, y_search, groups=train_df["unit"].values))

class CrossValidation:
    """Chấm mỗi cấu hình bằng kiểm định chéo, mỗi phần ba hạt giống."""
    def run_trial(self, trial, X, y, folds, **kwargs):
        hp = trial.hyperparameters
        scores = []
        for seed in SEEDS:
```

```
            for fit_index, hold_index in folds:
                fold_scaler = StandardScaler().fit(X[fit_index])
                X_fit = fold_scaler.transform(X[fit_index])
                X_hold = fold_scaler.transform(X[hold_index])
                keras.utils.set_random_seed(seed)
                model = self.hypermodel.build(hp)
                model.fit(X_fit, y[fit_index], verbose=0,
                          validation_data=(X_hold, y[hold_index]),
                          epochs=100, batch_size=128,
                          callbacks=[keras.callbacks.EarlyStopping(
                              monitor="val_loss", patience=10,
                              restore_best_weights=True)])
                scores.append(model.evaluate(
                    X_hold, y[hold_index], verbose=0)[1])
        return {"cv_rmse": float(np.mean(scores)),
                "cv_std": float(np.std(scores))}

class CVGridSearch(CrossValidation, kt.GridSearch):
    pass

class CVRandomSearch(CrossValidation, kt.RandomSearch):
    pass

def trial_table(tuner, columns):
    """Bảng các cấu hình đã thử, xếp theo RMSE tăng dần."""
    rows = []
    for trial in tuner.oracle.trials.values():
        row = {name: trial.hyperparameters.get(name)
               for name in columns}
        row["CV RMSE (chu trình)"] = round(
            trial.metrics.get_last_value("cv_rmse"), 2)
        row["Độ lệch chuẩn (±)"] = round(
            trial.metrics.get_last_value("cv_std"), 2)
        rows.append(row)
    table = pd.DataFrame(rows).sort_values(
        "CV RMSE (chu trình)")
    return table.reset_index(drop=True)

def describe_best(tuner, names):
    """In cấu hình và kết quả tốt nhất của một lần tìm kiếm."""
    trial = tuner.oracle.get_best_trials(1)[0]
    print("Cấu hình tốt nhất:")
    for name in names:
        value = trial.hyperparameters.get(name)
        text = f"{value:.3g}" if isinstance(value, float) else value
        print(f"  {name:<14}{text}")
    print("CV RMSE: "
          f"{trial.metrics.get_last_value('cv_rmse'):.2f} ± "
          f"{trial.metrics.get_last_value('cv_std'):.2f} chu trình")
```

6.6. Tìm kiếm lưới (Grid Search)

Ô này minh hoạ tìm kiếm lưới với hai siêu tham số, mỗi siêu tham số ba giá trị: số nơ-ron lớp ẩn đầu (32, 64, 128) và tốc độ học (10^{-3}, $3{\cdot}10^{-3}$, 10^{-2}), tức 3 × 3 = 9 cấu hình. Đối tượng `kt.HyperParameters` khai báo hai siêu tham số này bằng `Choice`; đối số `tune_new_entries=False` khiến bốn siêu tham số còn lại giữ giá trị mặc định ở bảng trên (hai lớp ẩn, Dropout 0,1, L2 10^{-4}, không BatchNormalization). `CVGridSearch.search` (với `verbose=0` để không in tiến trình từng lần thử; đổi thành `verbose=1` nếu muốn theo dõi) chấm từng cấu hình

bằng kiểm định chéo với ba hạt giống (12 lần huấn luyện mỗi cấu hình, 108 lần cho cả lưới). Ô in bảng chín cấu hình xếp theo RMSE tăng dần, rồi gọi `describe_best` để chỉ ra cấu hình tốt nhất (đủ sáu siêu tham số, kể cả bốn siêu tham số giữ mặc định) và điểm kiểm định chéo của nó. Kết quả chính: cấu hình tốt nhất là hai lớp ẩn 128–64 (số nơ-ron lớp đầu 128), tốc độ học 0,01, Dropout 0,1, L2 10^{-4} và không BatchNormalization, đạt 18,21 ± 0,93 chu trình. Điểm số của cả chín cấu hình chỉ trải từ 18,21 đến 18,74 chu trình, hẹp hơn độ lệch chuẩn của một cấu hình (0,8–1,2), nên chưa thể coi cấu hình thứ nhất tốt hơn hẳn các cấu hình khác. Xu hướng có thể đọc được là mạng 128 nơ-ron nhỉnh hơn mạng 32 nơ-ron ở cùng tốc độ học; còn tốc độ học 0,01 cho kết quả kém nhất ở mạng 32 và 64 nơ-ron nhưng tốt nhất ở mạng 128 nơ-ron, nên không có xu hướng nhất quán theo tốc độ học.

In [ ]:

```
# Lưới 3 x 3 = 9 cấu hình: số nơ-ron lớp ẩn đầu x tốc độ học
hp_grid = kt.HyperParameters()
hp_grid.Choice("units_1", [32, 64, 128])
hp_grid.Choice("learning_rate", [1e-3, 3e-3, 1e-2])

grid_tuner = CVGridSearch(
    hypermodel=build_model,
    objective=kt.Objective("cv_rmse", "min"),
    hyperparameters=hp_grid, tune_new_entries=False,
    max_trials=9, overwrite=True,
    directory="tuner", project_name="grid")
grid_tuner.search(X_search, y_search, folds, verbose=0)  # verbose=1: xem tiến trình

display(trial_table(grid_tuner, ["units_1", "learning_rate"]))
describe_best(grid_tuner, SPACE_NAMES)
```

	units_1	learning_rate	CV RMSE (chu trình)	Độ lệc
0	128	0.010	18.21	
1	128	0.003	18.22	
2	128	0.001	18.25	
3	64	0.003	18.35	
4	64	0.001	18.44	
5	32	0.003	18.52	
6	64	0.010	18.53	
7	32	0.001	18.56	
8	32	0.010	18.74	

```
Cấu hình tốt nhất:
  num_layers    2
  units_1       128
  learning_rate 0.01
  dropout       0.1
  l2            0.0001
  batch_norm    False
CV RMSE: 18.21 ± 0.93 chu trình
```

6.7. Tìm kiếm ngẫu nhiên (Random Search)

Ô này dùng tìm kiếm ngẫu nhiên trên cả sáu siêu tham số của hàm `build_model`, khoảng giá trị như bảng ở trên. `CVRandomSearch` bốc ngẫu nhiên mười cấu hình (`max_trials=10`, `seed` cố định để lặp lại được) và chấm từng cấu hình bằng cùng kiểm định chéo với ba hạt giống như ở mục 6.6, nên điểm số của hai phương án so sánh được với nhau. Ô in bảng mười cấu hình xếp theo RMSE tăng dần, rồi gọi `describe_best` để chỉ ra cấu hình tốt nhất và điểm kiểm định chéo của nó;

`get_best_hyperparameters` lấy cấu hình này để dùng ở mục sau. Kết quả chính: cấu hình tốt nhất là hai lớp ẩn 64–32, tốc độ học $3{\cdot}10^{-4}$, Dropout 0,1, L2 $2{,}7{\cdot}10^{-4}$ và không BatchNormalization, đạt 18,56 ± 0,84 chu trình, kém lưới ở mục 6.6 (18,21 chu trình) 0,35 chu trình, cùng bậc với độ lệch chuẩn; với chỉ mười lần thử trên không gian sáu chiều, tìm kiếm ngẫu nhiên ở lần chạy này không bốc trúng cấu hình tốt như lưới. Ba cấu hình đứng đầu chỉ cách nhau 0,23 chu trình. Điểm số trải từ 18,56 đến 33,39 chu trình; hai cấu hình thấp nhất (29,67 và 33,39) đều có tốc độ học rất thấp (dưới $2{\cdot}10^{-4}$) trên mạng nhỏ, nên mạng học quá chậm và chưa kịp hội tụ.

In [ ]:

```
# Tìm kiếm ngẫu nhiên: cả sáu siêu tham số, 10 cấu hình
random_tuner = CVRandomSearch(
    hypermodel=build_model,
    objective=kt.Objective("cv_rmse", "min"),
    max_trials=10, seed=0, overwrite=True,
    directory="tuner", project_name="random")
random_tuner.search(X_search, y_search, folds, verbose=0)

display(trial_table(random_tuner, SPACE_NAMES))
describe_best(random_tuner, SPACE_NAMES)
best_hp = random_tuner.get_best_hyperparameters(1)[0]
```

	num_layers	units_1	learning_rate	dropout	l2	batch_norm	CV RMSE (chu trình)	Độ lệc
0	2	64	0.000309	0.1	0.000267	False	18.56	
1	1	32	0.004734	0.0	0.000002	True	18.67	
2	1	128	0.000236	0.3	0.000001	False	18.79	
3	1	64	0.000440	0.3	0.000490	False	18.88	
4	1	16	0.004225	0.1	0.000004	True	18.95	
5	1	32	0.001048	0.3	0.000004	False	19.10	
6	3	32	0.000216	0.1	0.000013	False	19.11	
7	1	64	0.002058	0.2	0.000007	True	19.54	
8	2	16	0.000196	0.2	0.000002	True	29.67	
9	1	32	0.000119	0.2	0.000003	False	33.39	

```
Cấu hình tốt nhất:
  num_layers    2
  units_1       64
  learning_rate 0.000309
  dropout       0.1
  l2            0.000267
  batch_norm    False
CV RMSE: 18.56 ± 0.84 chu trình
```

6.8. Huấn luyện lại cấu hình tốt nhất

Ô này huấn luyện lại cấu hình tốt nhất của tìm kiếm ngẫu nhiên, `build_model(best_hp)`, với ba hạt giống trên toàn bộ tập huấn luyện, dùng tập kiểm định để dừng sớm (tối đa 200 epoch, patience 10 như ở mục 4). Hàm `train_final` trả về mạng đã huấn luyện và số epoch đã chạy; ô in số tham số và số epoch của từng hạt giống. `final_model` là mạng của hạt giống 2, dùng ở mục 7. Kết quả chính: mạng có 3265 tham số (so với 153 của mạng cơ sở) và dừng sớm ở epoch 64, 92 và 100 theo từng hạt giống.

In [ ]:

```
def train_final(seed):
    keras.utils.set_random_seed(seed)
```

```
    model = build_model(best_hp)
    history = model.fit(
        X_train, y_train, validation_data=(X_val, y_val),
        epochs=200, batch_size=64, verbose=0,
        callbacks=[keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=10,
            restore_best_weights=True)])
    return model, len(history.history["loss"])

final_runs = [train_final(seed) for seed in SEEDS]
final_models = [model for model, _ in final_runs]
final_model = final_models[0]                    # hạt giống 2
print("Số tham số:", final_model.count_params())
print("Số epoch:", [epochs for _, epochs in final_runs])
```

```
Số tham số: 3265
Số epoch: [64, 92, 100]
```

6.9. Đánh giá cấu hình tốt nhất trên tập kiểm tra

Đây là lần duy nhất dùng tập kiểm tra. Hàm `test_scores` tính RMSE và MAE trên tập kiểm tra (chu trình cuối của 100 động cơ) cho từng mạng. Ô so sánh hai nhóm mạng, mỗi nhóm ba hạt giống: mạng cơ sở ở mục 4 (`runs` ở mục 4.5) và mạng cấu hình tốt nhất vừa huấn luyện lại, báo cáo bằng trung bình ± độ lệch chuẩn. Kết quả chính: trên tập kiểm tra, mạng cơ sở đạt RMSE 17,35 ± 0,23 và MAE 12,27 ± 0,20 chu trình; cấu hình cuối đạt RMSE 17,09 ± 0,21 và MAE 11,96 ± 0,21 chu trình, giảm khoảng 0,26 chu trình (khoảng 1,5 %). Mức giảm cùng bậc với độ lệch chuẩn qua ba hạt giống, nhỏ so với sai số tuyệt đối (RMSE gần 17 chu trình) và chỉ dựa trên 100 động cơ kiểm tra, nên chưa thể coi là cải thiện thật sự; cần nhiều hạt giống hơn để khẳng định.

In [ ]:

```
# Lần DUY NHẤT dùng tập kiểm tra: mạng cơ sở và cấu hình cuối
baseline_models = [model for model, _ in runs]   # từ mục 4

def test_scores(models):
    rmses, maes = [], []
    for m in models:
        y_pred = m.predict(X_test, verbose=0).ravel()
        rmses.append(rmse(y_test, y_pred))
        maes.append(float(np.mean(np.abs(y_pred - y_test))))
    return rmses, maes

for name, models in (("Mạng cơ sở", baseline_models),
                     ("Cấu hình cuối", final_models)):
    rmses, maes = test_scores(models)
    print(f"{name:<14} RMSE {statistics.mean(rmses):.2f} ± "
          f"{statistics.pstdev(rmses):.2f} | MAE "
          f"{statistics.mean(maes):.2f} ± "
          f"{statistics.pstdev(maes):.2f} chu trình")
```

```
Mạng cơ sở     RMSE 17.35 ± 0.23 | MAE 12.27 ± 0.20 chu trình
Cấu hình cuối  RMSE 17.09 ± 0.21 | MAE 11.96 ± 0.21 chu trình
```

6.10. Tổng hợp: bảng năm phương án

Ô này tổng hợp năm phương án thành một bảng cùng định dạng với bảng cuối mục 5, theo trình tự cải tiến: mạng nông 17–8–1 (200 epoch), mạng nông huấn luyện lâu (dừng sớm, tối đa 2000 epoch) mạng sâu 17–256–256–1 không chính quy hoá, mạng sâu chính quy hoá cả bốn kỹ thuật (L2, Dropout, dừng sớm và BatchNormalization) và mạng tinh chỉnh siêu tham số tốt nhất. Bốn phương án đầu lấy từ `RESULTS` của mục 5; kết quả ba hạt giống của mạng tinh chỉnh (huấn luyện lại ở trên) được thêm vào `RESULTS` cùng định dạng rồi gom bằng `summarize`. Bảng gồm kiến trúc, RMSE huấn luyện, RMSE kiểm định (trung bình ± độ lệch chuẩn), độ chênh kiểm định – huấn luyện và số epoch trung bình; mọi số liệu trong bảng đo trên tập huấn luyện và tập kiểm định, còn kết quả trên tập kiểm tra nằm ở ô đánh giá phía trên. Kết quả chính: RMSE kiểm định không đổi đáng kể từ mạng nông (16,96 chu trình) sang mạng nông huấn luyện lâu (16,94); mạng sâu không chính quy hoá quá khớp (18,91, độ chênh +3,47); chính quy hoá cả bốn kỹ thuật đưa về 16,80 với độ chênh −1,09; mạng tinh chỉnh siêu tham số đạt 17,04 ± 0,10 chu trình, với 3265 tham số (so với 70.657 của mạng sâu) và 85 epoch, tức không tốt hơn mạng chính quy hoá cả bốn kỹ thuật. Số liệu kiểm định của mạng tinh chỉnh có thể hơi lạc quan, vì 20 động cơ kiểm định nằm trong dữ liệu dùng để chọn siêu tham số; con số dùng để kết luận là RMSE trên tập kiểm tra ở ô đánh giá phía trên (17,09 ± 0,21 chu trình).

In [ ]:

```
RESULTS["Tinh chỉnh siêu tham số"] = [
    dict(train=model.evaluate(X_train, y_train, verbose=0)[1],
         val=model.evaluate(X_val, y_val, verbose=0)[1],
         epochs=epochs)
    for model, epochs in final_runs]

best_sizes = layer_sizes(best_hp.get("num_layers"),
                         best_hp.get("units_1"))
FINAL_VARIANTS = [   # (khoá trong RESULTS, tên phương án, kiến trúc)
    ("17-8-1", "Mạng nông", "17-8-1"),
    ("17-8-1, huấn luyện lâu", "Mạng nông, huấn luyện lâu",
     "17-8-1"),
    ("17-256-256-1", "Mạng sâu, không chính quy hoá",
     "17-256-256-1"),
    ("+ cả bốn kỹ thuật", "Mạng sâu, chính quy hoá cả bốn kỹ thuật",
     "17-256-256-1"),
    ("Tinh chỉnh siêu tham số", "Mạng tinh chỉnh siêu tham số tốt nhất",
     "17-" + "-".join(map(str, best_sizes)) + "-1")]

final_table = pd.DataFrame([summarize(key)
                            for key, _, _ in FINAL_VARIANTS])
final_table["Phương án"] = [name for _, name, _ in FINAL_VARIANTS]
final_table.insert(1, "Kiến trúc",
                   [arch for _, _, arch in FINAL_VARIANTS])
display(final_table)
```

	Phương án	Kiến trúc	RMSE huấn luyện	RMSE kiểm định	Chênh (chu trình)
0	Mạng nông	17-8-1	18.35	16.96 ± 0.26	-1.39
1	Mạng nông, huấn luyện lâu	17-8-1	18.34	16.94 ± 0.24	-1.40
2	Mạng sâu, không chính quy hoá	17-256-256-1	15.44	18.91 ± 0.26	3.47
3	Mạng sâu, chính quy hoá cả bốn kỹ thuật	17-256-256-1	17.89	16.80 ± 0.06	-1.09
4	Mạng tinh chỉnh siêu tham số tốt nhất	17-64-32-1	18.52	17.04 ± 0.10	-1.48

7. Lưu và dùng lại mô hình

7.1. Phương pháp làm ở mục 7: lưu mô hình và dùng lại

Sau khi chọn được cấu hình tốt nhất, bước cuối cùng là lưu mô hình để dùng lại mà không phải huấn luyện lại. Mạng chỉ dùng được khi đi cùng cách xử lý dữ liệu đã dùng lúc huấn luyện, nên phải lưu cả hai: mạng `final_model` (tệp `.keras` chứa kiến trúc, trọng số và cấu hình biên dịch) và các thành phần tiền xử lý gồm bộ chuẩn hoá `scaler`, danh sách tên đặc trưng `FEATURES` và mức chặn `RUL_CAP` (lưu bằng `joblib`). Cả hai được lưu vào một thư mục trên Google Drive để còn nguyên sau khi phiên Colab kết thúc.

Khi dùng lại, ta nạp mạng bằng `keras.models.load_model` và tiền xử lý bằng `joblib.load`, kiểm tra mạng nạp lại cho dự đoán trùng với mạng gốc, rồi dự báo RUL trực tiếp từ số đọc cảm biến của chu trình mới nhất: lấy đúng các cột đặc trưng theo thứ tự đã lưu, chuẩn hoá bằng bộ chuẩn hoá đã lưu rồi đưa vào mạng. Danh sách đặc trưng phải được lưu cùng mạng, vì nếu thứ tự hoặc tập cột khác lúc huấn luyện thì dự đoán sẽ sai mà không báo lỗi.

7.2. Lưu mô hình

Ô này khai báo thư mục `MODEL_DIR` trên Drive (Drive đã được kết nối ở mục 2), tạo thư mục nếu chưa có, rồi lưu hai tệp. Tệp `rul_mlp.keras` là mạng của hạt giống 2 trong cấu hình tốt nhất, lưu bằng `final_model.save`. Tệp `rul_preprocess.joblib` là một từ điển gồm `scaler`, `features` và `rul_cap`, lưu bằng `joblib.dump`. Ô in danh sách tệp trong thư mục để xác nhận đã lưu.

In [ ]:

```
import os
import joblib

MODEL_DIR = "/content/gdrive/MyDrive/Colab Notebooks/Model"  # thư mục trên Drive
os.makedirs(MODEL_DIR, exist_ok=True)

# Lưu CẢ HAI: mạng và các thành phần tiền xử lý
final_model.save(f"{MODEL_DIR}/rul_mlp.keras")
joblib.dump({"scaler": scaler, "features": FEATURES,
             "rul_cap": RUL_CAP},
            f"{MODEL_DIR}/rul_preprocess.joblib")

print(sorted(os.listdir(MODEL_DIR)))
```

```
['rul_mlp.keras', 'rul_preprocess.joblib']
```

7.3. Gọi mô hình đã lưu từ Drive để dùng lại

Phần này mô phỏng việc dùng lại mô hình sau khi huấn luyện xong. Trong một phiên mới chỉ cần kết nối Drive, nạp `keras`, `numpy`, `pandas` và `joblib` rồi chạy hai ô dưới đây; không cần dữ liệu huấn luyện, không cần khai báo lại hàm nào và không cần huấn luyện lại.

7.3.1. Nạp mô hình và kiểm tra dự đoán trùng khớp

Ô này nạp mạng bằng `keras.models.load_model` và các thành phần tiền xử lý bằng `joblib.load` từ `MODEL_DIR`. Để chắc chắn việc lưu và nạp không làm đổi mô hình, ô cho cả mạng nạp lại và mạng gốc dự đoán trên tập kiểm tra rồi so sánh bằng `np.allclose`; kết quả `True` nghĩa là hai mạng cho dự đoán trùng nhau. Kết quả: `True`.

In [ ]:

```
# Nạp lại mạng và tiền xử lý đã lưu trên Drive
loaded_model = keras.models.load_model(f"{MODEL_DIR}/rul_mlp.keras")
preprocess = joblib.load(f"{MODEL_DIR}/rul_preprocess.joblib")

# Mạng nạp lại phải cho dự đoán trùng mạng gốc
same = np.allclose(loaded_model.predict(X_test, verbose=0),
                   final_model.predict(X_test, verbose=0))
print("Dự đoán của mạng nạp lại trùng mạng gốc:", same)
```

```
Dự đoán của mạng nạp lại trùng mạng gốc: True
```

7.3.2. Dự báo RUL từ số đọc cảm biến của chu trình mới nhất

Hàm `predict_rul` nhận số đọc cảm biến của chu trình mới nhất (một dòng của bảng, có tên cột) và trả về RUL dự báo: lấy các cột theo `preprocess["features"]`, chuẩn hoá bằng `preprocess["scaler"]`, đưa vào `loaded_model`, rồi giới hạn kết quả trong khoảng từ 0 đến `preprocess["rul_cap"]` vì nhãn lúc huấn luyện không âm và bị chặn ở mức đó. Ô gọi hàm này cho mười động cơ đầu tiên của tập kiểm tra (chu trình cuối của mỗi động cơ, trong `test_last`), rồi gom kết quả vào một bảng `pandas` gồm số hiệu động cơ, RUL thực tế, RUL dự báo và sai số (dự báo trừ thực tế). Mười động cơ chỉ minh hoạ cách gọi mô hình, không thay cho việc đánh giá trên cả 100 động cơ ở mục 6.9. Kết quả: 8 trong 10 dự báo cao hơn thực tế, sai số trung bình +5,7 chu trình, lớn nhất +17,4 chu trình (động cơ 2); hai động cơ bị dự báo thấp hơn là động cơ 8 (−3,2) và động cơ 9 (−12,1). Mười động cơ này đều có RUL thực tế từ 69 đến 112 chu trình nên bảng chỉ minh hoạ cách gọi, không đại diện cho toàn bộ tập kiểm tra.

In [ ]:

```
def predict_rul(sensor_row):
    """Số đọc ở chu trình mới nhất (một dòng có tên cột) -> RUL dự báo."""
    row = pd.DataFrame([sensor_row])[preprocess["features"]]
    features = preprocess["scaler"].transform(row)
    rul = float(loaded_model.predict(features, verbose=0)[0, 0])
    return float(np.clip(rul, 0, preprocess["rul_cap"]))

# Mười động cơ đầu tiên của tập kiểm tra, chu trình cuối của mỗi động cơ
rows = []
for index in range(10):
    engine = test_last.iloc[index]
    predicted = predict_rul(engine)
```

```
    rows.append({"Động cơ": int(engine["unit"]),
                 "RUL thực tế": int(y_test[index]),
                 "RUL dự báo": round(predicted, 1),
                 "Sai số (dự báo − thực tế)":
                     round(predicted - y_test[index], 1)})
display(pd.DataFrame(rows))
```

	Động cơ	RUL thực tế	RUL dự báo	Sai số (dự bá
0	1	112	123.7	
1	2	98	115.4	
2	3	69	71.3	
3	4	82	93.9	
4	5	91	105.6	
5	6	93	101.3	
6	7	91	96.1	
7	8	95	91.8	
8	9	111	98.9	
9	10	96	97.1	

8. Tổng kết quy trình

Quy trình sáu bước được áp dụng liên tiếp cho bài toán dự báo tuổi thọ còn lại của động cơ bằng mạng MLP.

Mục	Việc đã làm	Đầu ra chính
2. Xác định bài toán, dữ liệu	Đọc ba tệp C-MAPSS FD001, quan sát diễn biến cảm biến, xác định bài toán hồi quy RUL	Bài toán và dữ liệu rõ ràng
3. Chia tập và tiền xử lý	Tạo nhãn RUL chặn ở 125, chọn 17 đặc trưng, chia ba tập theo động cơ, chuẩn hoá chỉ trên tập huấn luyện	Ba tập không rò rỉ thông tin và bộ chuẩn hoá `scaler`
4. Khai báo và huấn luyện mạng cơ bản	Dựng mạng 17–8–1, khai báo Adam, mất mát `mse`, RMSE, MAE và dừng sớm; huấn luyện với một hạt giống rồi ba hạt giống, vẽ đường học và đồ thị RUL dự báo – thực tế	Mạng `model_bs`, mốc so sánh, biết mạng chưa khớp hay quá khớp và sai số theo khoảng RUL
5. Cải tiến theo thử và sai	Huấn luyện lâu hơn, tăng kích thước mạng, thêm kỹ thuật chính quy hoá	Bảng so sánh tám phương án
6. Tinh chỉnh siêu tham số	Tìm kiếm lưới và tìm kiếm ngẫu nhiên với kiểm định chéo theo động cơ, huấn luyện lại, đánh giá một lần trên tập kiểm tra	Cấu hình tốt nhất và bảng so sánh năm phương án
7. Lưu và dùng lại mô hình	Lưu mạng cùng tiền xử lý, nạp lại từ Drive để dự báo RUL	Mô hình dùng lại được mà không cần huấn luyện lại

Quy trình lặp lại một vòng: huấn luyện, chẩn đoán, thay đổi một yếu tố, đo lại. Bốn nguyên tắc đi xuyên suốt cả quy trình. Dữ liệu chia theo động cơ và bộ chuẩn hoá chỉ khớp trên tập huấn luyện để không rò rỉ thông tin. Mỗi mạng được huấn luyện với nhiều hạt giống và báo cáo bằng trung bình ± độ lệch chuẩn, vì chênh lệch nhỏ có thể chỉ do khởi tạo ngẫu nhiên. Mỗi lần chỉ đổi một yếu tố để biết yếu tố nào có tác dụng. Tập kiểm tra chỉ được dùng một lần ở cuối, và khi lưu mô hình phải lưu cả mạng lẫn tiền xử lý.